



PERÍODO: 202650 Abril-Agosto 2026
NOMBRE: Pilaguano Chisaguano David Alexander
CARRERA: Ingeniería en software

PARCIAL: 1
CURSO (NRC): 30723
FECHA: 2026/05/07

Examen Unidad 1 - Análisis y Diseño de Software

Forma A: Modelos de análisis con PlantUML a partir de requisitos funcionales consecutivos

1. Contexto base del examen

Caso de estudio: Mueblerix. El examen se fundamenta en la lógica de modelado de la guía gamificada de PlantUML, donde los estudiantes transforman requisitos funcionales en modelos UML claros, trazables y coherentes. Para esta Forma A se tomarán dos requisitos funcionales consecutivos del documento de especificación: RF-03 Añadir Producto y RF-04 Consultar Producto.

2. Requisitos funcionales seleccionados

RF	Actor	Propósito	Precondición principal	Resultado esperado
RF-03 Añadir Producto	Administrador	Registrar un nuevo producto en el catálogo con nombre, precio, categoría, material, color e imágenes.	Administrador autenticado y sistema en funcionamiento.	Producto registrado y disponible en el catálogo.
RF-04 Consultar Producto	Administrador	Buscar, filtrar y visualizar productos del catálogo por nombre, categoría, material, color o rango de precio.	Administrador autenticado y al menos un producto registrado.	Productos coincidentes mostrados correctamente para consulta.

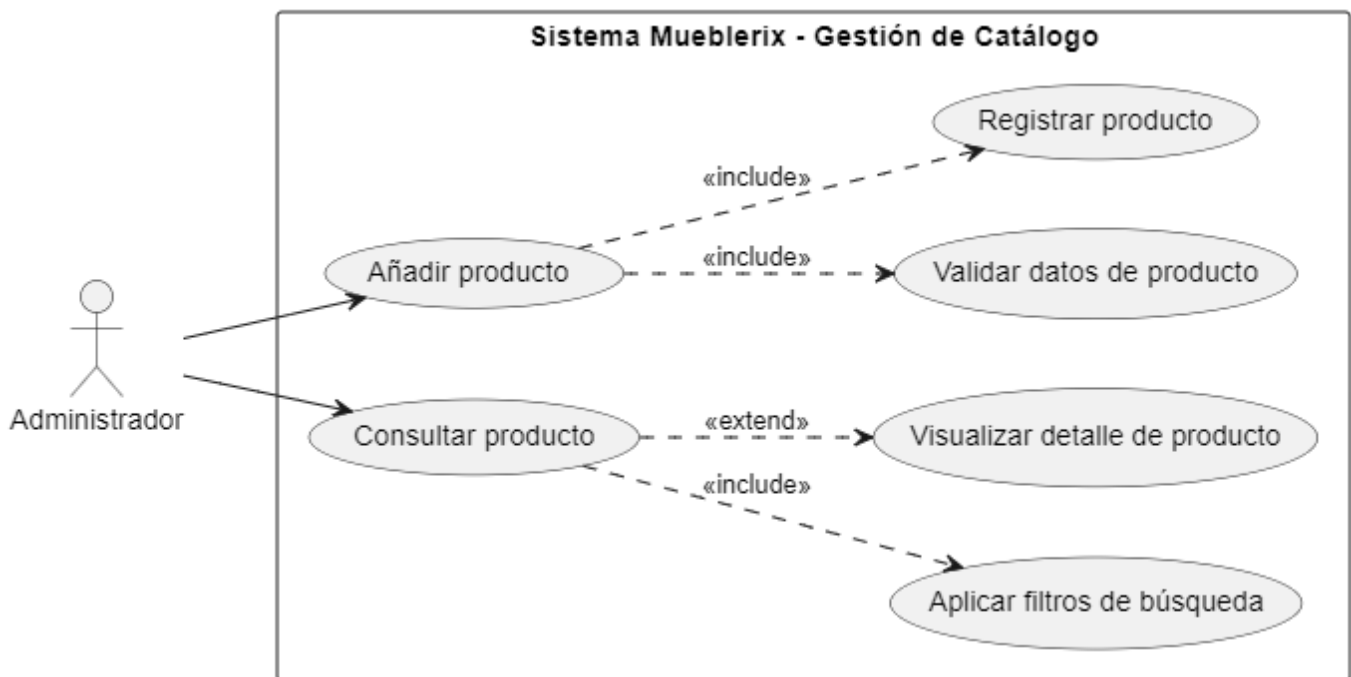
3. Instrucciones para el estudiante

- Lea los requisitos RF-03 y RF-04 y modele únicamente lo necesario para comprender el análisis del dominio.
- Construya los artefactos con sintaxis PlantUML: un diagrama de casos de uso de análisis, un diagrama de clases de análisis y dos diagramas de secuencia de análisis.
- Use nombres de casos de uso con verbo en infinitivo y evite convertir pantallas, botones o bases de datos en casos de uso.
- Justifique brevemente las decisiones de modelado, especialmente las relaciones entre casos de uso, clases y mensajes de secuencia.
- No modele detalles de implementación como frameworks, API REST, archivos físicos, controladores técnicos o interfaces gráficas específicas.

4. Modelo de casos de uso de análisis y Puente

El actor principal es el Administrador. Los casos de uso centrales son Añadir producto y Consultar producto. Se incorporan relaciones <<include>> cuando la acción es obligatoria para completar el objetivo del caso de uso.

```
@startuml
left to right direction
actor Administrador
rectangle "Sistema Mueblerix - Gestión de Catálogo" {
    usecase "Añadir producto" as UC_Agregar
    usecase "Consultar producto" as UC_Consultar
    usecase "Validar datos de producto" as UC_Validar
    usecase "Registrar producto" as UC_Registrar
    usecase "Aplicar filtros de búsqueda" as UC_Filtrar
    usecase "Visualizar detalle de producto" as UC_Detalle
}
Administrador --> UC_Agregar
Administrador --> UC_Consultar
UC_Agregar ..> UC_Validar : <<include>>
UC_Agregar ..> UC_Registrar : <<include>>
UC_Consultar ..> UC_Filtrar : <<include>>
UC_Consultar ..> UC_Detalle : <<extend>>
@enduml
```



5. Diagrama de clases de análisis

El modelo separa clases de frontera, control y entidad para evidenciar responsabilidades de análisis: interacción del administrador, coordinación de reglas y objetos propios del dominio del catálogo.

```
@startuml
left to the right direction
skinparam classAttributeIconSize 0

package "Capa Frontera" {
    class "InterfazGestionCatalogo" <<boundary>> {

```

```

+ mostrarFormularioProducto()
+ mostrarFiltrosBusqueda()
+ mostrarResultados(lista)
}
}

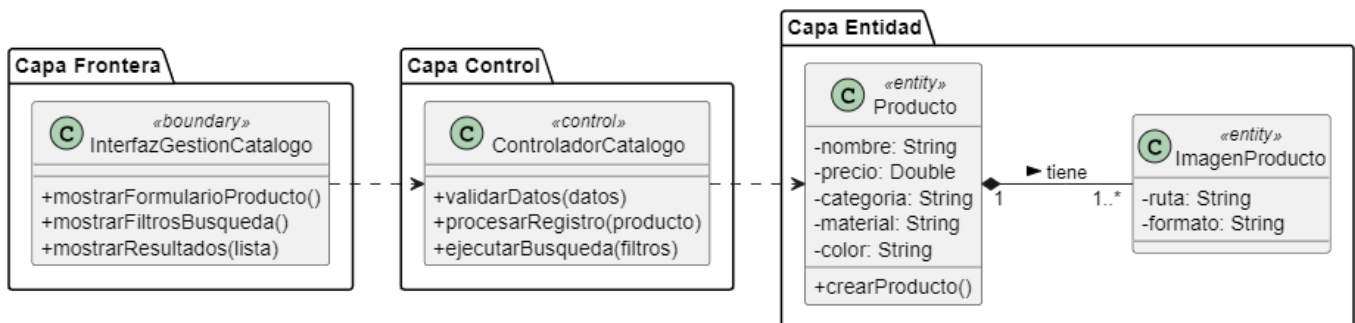
package "Capa Control" {
class "ControladorCatalogo" <<control>> {
+ validarDatos(datos)
+ procesarRegistro(producto)
+ ejecutarBusqueda(filtros)
}
}

package "Capa Entidad" {
class "Producto" <<entity>> {
- nombre: String
- precio: Double
- categoria: String
- material: String
- color: String
+ crearProducto()
}

class "ImagenProducto" <<entity>> {
- ruta: String
- formato: String
}
}

InterfazGestionCatalogo ..> ControladorCatalogo
ControladorCatalogo ..> Producto
Producto "1" *-- "1..*" ImagenProducto : tiene >
@enduml

```



6. Diagrama de secuencia de análisis 1: RF-03 Añadir Producto

```

@startuml
actor Administrador
participant "InterfazGestionCatalogo" as UI <<boundary>>
participant "ControladorCatalogo" as Ctrl <<control>>
participant "Producto" as Entidad <<entity>>

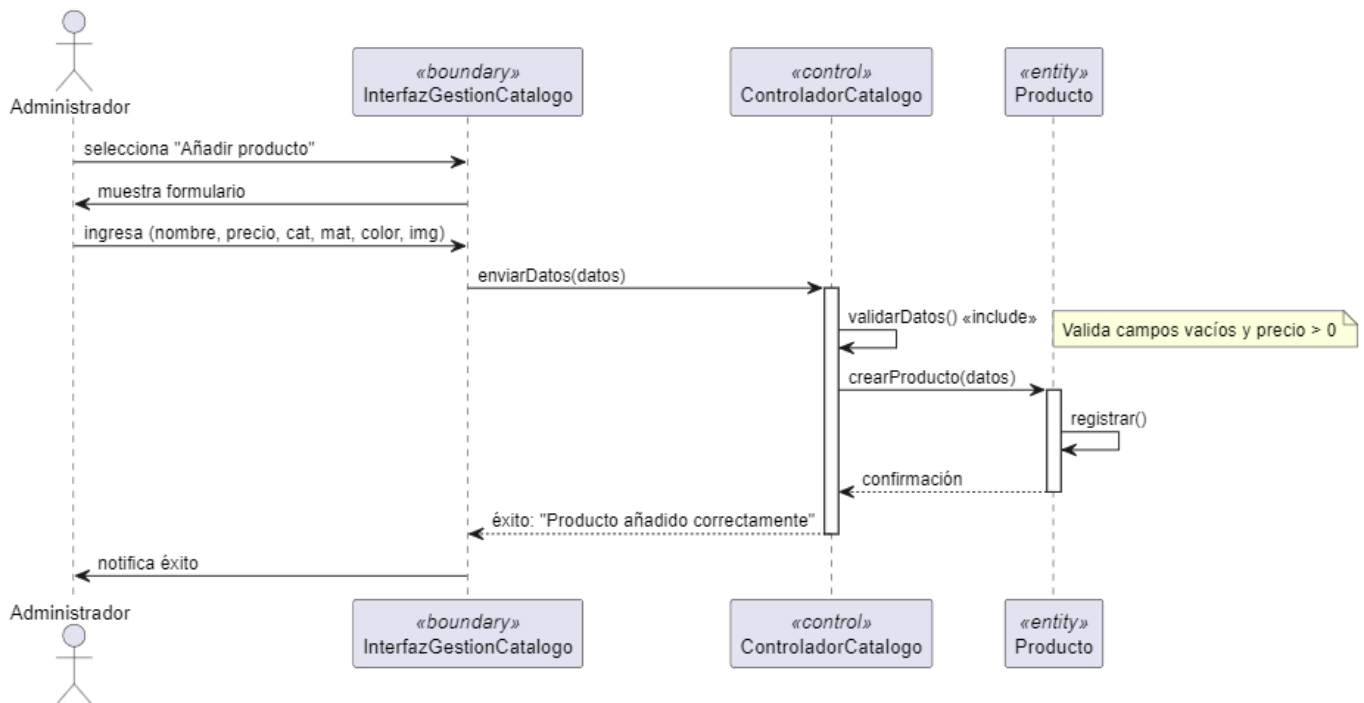
Administrador -> UI : selecciona "Añadir producto"
UI -> Administrador : muestra formulario
Administrador -> UI : ingresa (nombre, precio, cat, mat, color, img)

```

```

UI -> Ctrl : enviarDatos(datos)
activate Ctrl
Ctrl -> Ctrl : validarDatos() <<include>>
note right: Valida campos vacíos y precio > 0
Ctrl -> Entidad : crearProducto(datos)
activate Entidad
Entidad -> Entidad : registrar()
Entidad --> Ctrl : confirmación
deactivate Entidad
Ctrl --> UI : éxito: "Producto añadido correctamente"
deactivate Ctrl
UI -> Administrador : notifica éxito
@enduml

```



7. Diagrama de secuencia de análisis 2: RF-04 Consultar Producto

```

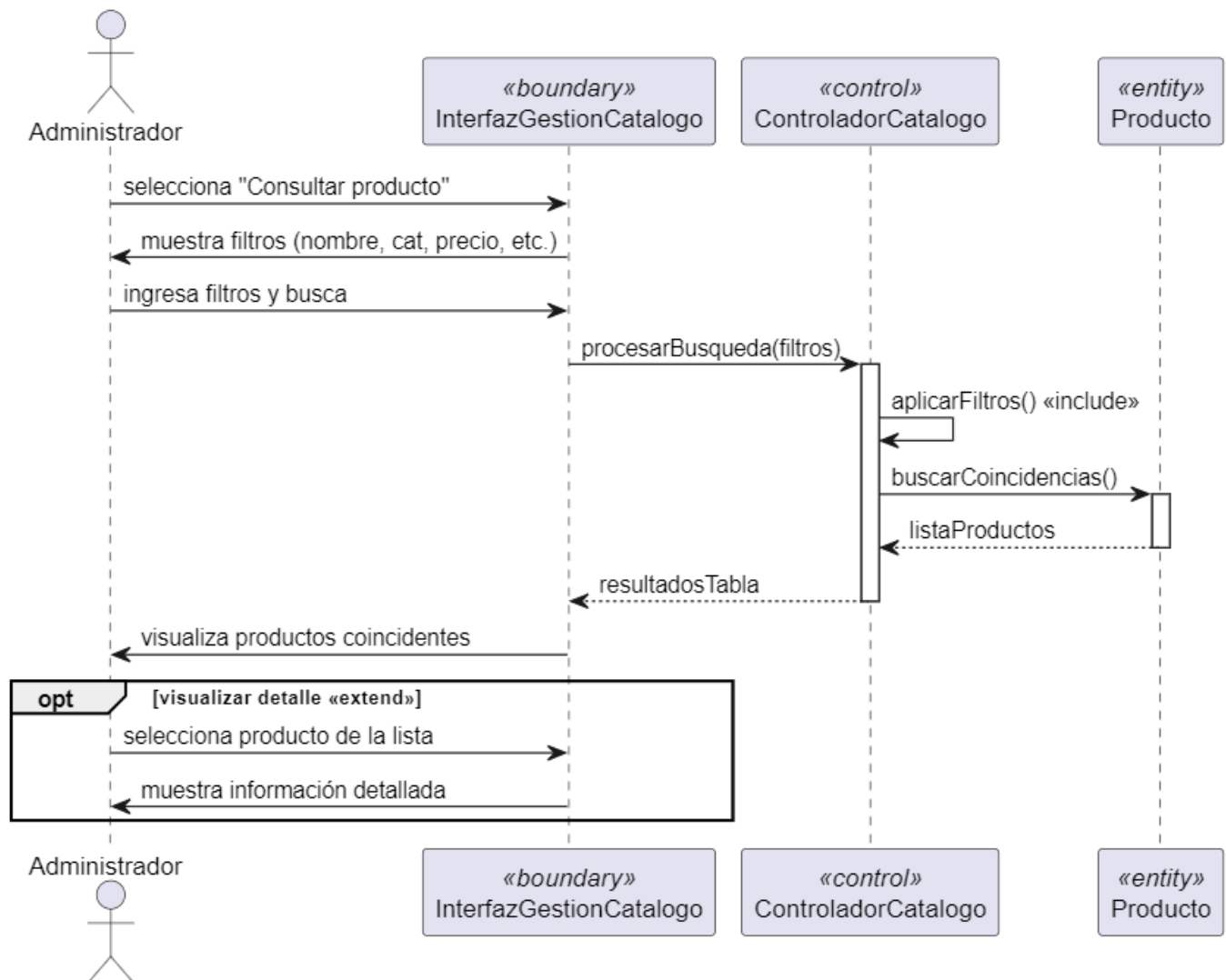
@startuml
actor Administrador
participant "InterfazGestionCatalogo" as UI <<boundary>>
participant "ControladorCatalogo" as Ctrl <<control>>
participant "Producto" as Entidad <<entity>>

Administrador -> UI : selecciona "Consultar producto"
UI -> Administrador : muestra filtros (nombre, cat, precio, etc.)
Administrador -> UI : ingresa filtros y busca
UI -> Ctrl : procesarBusqueda(filtros)
activate Ctrl
Ctrl -> Ctrl : aplicarFiltros() <<include>>
Ctrl -> Entidad : buscarCoincidencias()

```

activate Entidad
Entidad --> Ctrl : listaProductos
deactivate Entidad
Ctrl --> UI : resultadosTabla
deactivate Ctrl
UI -> Administrador : visualiza productos coincidentes

opt visualizar detalle <<extend>>
Administrador -> UI : selecciona producto de la lista
UI -> Administrador : muestra información detallada
end
@enduml



8. Preguntas de pensamiento crítico para justificar el análisis

1. Explique por qué RF-03 y RF-04 pueden formar parte del mismo límite del sistema “Gestión de Catálogo”.

Respuesta: RF-03 y RF-04 forman parte del mismo límite del sistema «Gestión de Catálogo» porque ambos tienen el mismo objetivo, el cual es mantener el inventario digital de la mueblería, y operan sobre las mismas entidades (productos y sus atributos), todo esto bajo la responsabilidad del administrador como actor.

2. Justifique la relación <<include>> entre Añadir producto y Validar datos de producto.

Respuesta: El «include» en la relación de «Añadir producto» y «Validar datos» se debe a que la validación es una acción obligatoria por parte del sistema, ya que un producto no puede ni debe ser registrado en una base de datos sin verificar que los campos obligatorios estén llenos y que el precio sea válido en este caso mayor a 0

3. Argumente por qué ImagenProducto se modela con composición respecto a Producto.

Respuesta: Se usa ImagenProducto porque la existencia de la imagen depende completamente del producto, si un producto es eliminado del catalogo, sus imagenes pierden el sentido dentro del Sistema, y como Sistema de validación un producto requiere al menos una imagen para ser válido

4. Identifique un riesgo de análisis si se confunde una pantalla del sistema con un caso de uso.

Respuesta: El riesgo de confundir una pantalla con un caso de uso es que se puede omitir reglas de negocio necesarias o validaciones que ocurren detras de una interfaz

5. Explique cómo la trazabilidad entre requisitos, casos de uso, clases y secuencias permite detectar errores tempranos.

Respuesta: La trazabilidad nos permite asegurar que cada mensaje y atributo en una secuencia tenga su origen en un requisito funcional, en caso de que un paso en la secuencia no exista en el requisito se da una inconsistencia, otro ejemplo es con ImagenProductor, si la clase producto no contempla las imagenes aunque el requisito las pide entonces existe una omission antes de la programación

9. Justificación breve de Trazabilidad

9. Rúbrica de evaluación sobre 20 puntos

Criterio	Excelente	Bueno	Básico	Insuficiente	Puntaje
1. Interpretación y trazabilidad de requisitos	Relaciona RF-03 y RF-04 con objetivos, precondiciones y resultados sin perder trazabilidad.	Reconoce la mayoría de elementos, con leves omisiones.	Identifica elementos generales, pero con trazabilidad débil.	Confunde requisito, caso de uso y funcionalidad técnica.	4
2. Casos de uso y relaciones	Define actor, límite del sistema, casos de uso y relaciones <<include>> / <<extend>> con sentido analítico.	Presenta casos claros, aunque alguna relación requiere mejor justificación.	Incluye casos de uso, pero con nombres o relaciones poco precisos.	Modela pantallas o acciones internas como casos de uso.	4
3. Clases de análisis del dominio	Selecciona clases relevantes, atributos esenciales y relaciones coherentes con el catálogo.	Clases adecuadas con alguna relación o multiplicidad mejorable.	Clases incompletas o mezcladas con elementos técnicos.	No representa el dominio o copia términos sin analizar.	4
4. Secuencias de análisis	Los mensajes reflejan responsabilidades, validaciones y flujos alternos de los dos RF.	La secuencia es comprensible, con leves vacíos en validación o retorno.	Secuencia lineal con poca responsabilidad entre objetos.	Mensajes incoherentes o sin relación con los RF.	4
5. Pensamiento crítico y justificación	Argumenta decisiones, límites, riesgos y coherencia entre modelos.	Justifica decisiones principales, aunque con poca profundidad.	Explica de forma descriptiva, con escaso análisis crítico.	No justifica o presenta respuestas memorísticas.	4

10. Entregables esperados del estudiante en un solo document con sus Apellidos y Nombres

- Código PlantUML del diagrama de casos de uso de análisis y puente.
- Código PlantUML del diagrama de clases de análisis.
- Código PlantUML de dos diagramas de secuencia: Añadir Producto y Consultar Producto.
- Respuestas a las preguntas de pensamiento crítico.
- Justificación breve de trazabilidad entre RF, CU, clases y secuencias.

Justificación de la trazabilidad:

RF a CU: Los requisitos RF-03 y RF-04 se transforman directamente en los casos de uso centrales "Añadir producto" y "Consultar producto" ambos relacionados de manera directa ya que trabajan con las mismas entidades y con el mismo actor, en este caso el Administrador es el único con el acceso a estos requisitos, ya que mediante la precondición se asegura que solo tenga el acceso

CU a Secuencia: Los pasos de la "Secuencia Normal" descritos en los documentos de requisitos se mapean como mensajes entre los objetos Frontera, Control y Entidad en los diagramas de secuencia

RF a Clases: Los atributos definidos en la descripción del requisito (nombre, precio, material, color) se convierten en los atributos privados de la entidad Producto en el diagrama de clases

Mediante todos estos se puede obtener una trazabilidad completa, en donde no existen clases que no se hayan contemplado anteriormente o atributos que carezcan de validaciones, dando un completo seguimiento al proceso de estos requisitos